Automated route for this repository

The current project is Lanka Dengue Sequence Classifier. Use the new local-data report, not the previous Ecoli draft. This guide prepares deployment; the user's account must execute it.

After student login and local setup below, the shortest route from the project root is:

```
az login
az account list --output table
az account set --subscription 'REPLACE_WITH_YOUR_STUDENT_SUBSCRIPTION_ID'
bash scripts/deploy_azure.sh
```

The script creates a dedicated resource group, Basic registry, image build, pull identity, Container Apps environment and public HTTPS app; then it checks Streamlit health. It records resource names and hostname in deployment-state.json. Test the frontend manually after health succeeds. For a student subscription where ACR Tasks are unavailable, start Docker Desktop and use:

```
BUILD_MODE=docker bash scripts/deploy_azure.sh
```

To choose a permitted region before the first attempt:

```
LOCATION=eastus bash scripts/deploy_azure.sh
```

Default resource group: rg-lanka-dengue-assignment. Do not mix unrelated resources into it. Stable registry naming makes reruns reuse the intended registry. If a command fails, inspect the error and existing resources rather than repeatedly creating new groups. After marking, `bash scripts/cleanup_azure.sh` lists resources and asks you to type the exact group name before Azure's deletion confirmation.

The sections below explain each step from scratch and provide a manual alternative. Use either the automated route or the manual route consistently; their example resource names differ. Do not deploy both simply to follow the guide.

Azure deployment from scratch

Prepared 2 November 2026. Commands are for Bash, macOS Terminal or WSL; do not paste them into PowerShell unchanged. This is a deployment procedure, not evidence of an already deployed service. You need your own Azure sign-in. Allow 45-90 minutes the first time.

1. Sign in and verify student credits

1. Open https://portal.azure.com in your browser. Select Sign in. Use the Microsoft account linked to your existing student subscription; complete MFA privately. 2. If you have no activated offer, visit https://azure.microsoft.com/en-us/free/students/ and select Start free. Follow identity and student verification using your university email. Do not create a second offer if your existing credits are active. 3. In the portal search bar enter Subscriptions. Open your Azure for Students subscription. Check Status is Active and record its Subscription ID privately. If absent, use the profile menu's directory/subscription switcher and choose the directory containing it. Azure for Students Starter is a different offer and might not support this deployment. 4. Search Cost Management + Billing and inspect the subscription's remaining credit/expiry using its offer/credit link (some student offers use the Sponsorships portal). Check the actual remaining amount; do not assume the advertised initial credit is still available. 5. Under Cost Management > Budgets, create a small monthly budget in your billing currency (for example equivalent to US$5) with 50%, 80%, and 100% email alerts, if available. A budget is an alert, NOT a spending stop. New subscriptions can take time to expose cost data. 6. This architecture uses Container Apps Consumption, a Basic container registry, build tasks, and logs. The registry can incur idle charges and builds/logs can consume credits even when the app scales to zero. Check the portal cost estimate and current pricing; no zero-cost guarantee is made. Preserve the student spending limit; upgrading to paid billing is not required by this guide.

2. Prepare and run locally

Install Python 3.11 from https://www.python.org/downloads/ and VS Code if desired. Unzip the source archive. Open Terminal and cd into the folder containing app.py. Commands below assume your shell stays in that folder.

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python train.py
streamlit run app.py
```

On Windows use WSL Bash for this guide, or activate a Windows venv with `.venv\Scripts\Activate.ps1` in PowerShell only for local testing. Open http://localhost:8501. Choose a public held-out genome, click Classify and explain, check prediction, local sensitivity bars, and results. Stop with Ctrl+C. QDA collinearity warnings are documented: positive regularization stabilizes the estimate; they are not download failures.

3. Install Azure CLI and log in

Install using Microsoft's operating-system instructions: https://learn.microsoft.com/en-us/cli/azure/install-azure-cli . On a Mac with Homebrew already installed, `brew install azure-cli` is an option. A local Docker installation is not needed for the primary cloud-build route.

```
az version
az login
az account list --output table
az account set --subscription 'REPLACE_WITH_YOUR_SUBSCRIPTION_ID'
az account show --query '{Name:name,State:state}' --output table
az extension add --name containerapp --upgrade
```

Choose the student account in the browser sign-in. If browser login cannot launch, use `az login --use-device-code`; enter the one-time code only on Microsoft's official page. Do not show the code, MFA, account details or tokens in a recording.

4. Choose region, names and register services

Use the SAME terminal so variables persist. Southeast Asia is an example near Sri Lanka, not a guarantee of availability for student offers. If blocked, select a permitted Container Apps region shown by Azure, e.g. East US where allowed. Change LOCATION before creating resources.

```
RG=rg-lanka-dengue-demo
LOCATION=southeastasia
APP=lanka-dengue-demo
ENVIRONMENT=env-lanka-dengue-demo
ACR=lkdengue$(date +%s)
az provider register --namespace Microsoft.App --wait
az provider register --namespace Microsoft.OperationalInsights --wait
az provider register --namespace Microsoft.ContainerRegistry --wait
az provider register --namespace Microsoft.ManagedIdentity --wait
az group create --name "$RG" --location "$LOCATION"
```

ACR names must be globally unique, lowercase letters/digits only. Write down your chosen ACR value (not a secret). A lost terminal loses these variables; re-enter the same values rather than creating duplicate resources.

5. Build the application image in Azure

Run from the source folder, NOT its parent. Dockerfile installs pinned packages and runs train.py during image build, so the model uses the serving environment's Python version.

```
az acr create --resource-group "$RG" --name "$ACR" --sku Basic
az acr build --registry "$ACR" --image dengue-app:v1 .
LOGIN_SERVER=$(az acr show --name "$ACR" --query loginServer --output tsv)
```

Wait for a successful build. Read the build log if it fails. ACR Tasks can be unavailable on some student subscriptions. If the error explicitly says Tasks are unsupported, use the fallback in section 10. A successful cloud build is still needed to verify the Dockerfile; this package has only been tested locally so far.

6. Grant the app permission to pull its image

Use a managed identity, avoiding registry passwords.

```
az identity create --resource-group "$RG" --name dengue-pull
IDENTITY_ID=$(az identity show -g "$RG" -n dengue-pull --query id -o tsv)
PRINCIPAL_ID=$(az identity show -g "$RG" -n dengue-pull --query principalId -o tsv)
ACR_ID=$(az acr show -n "$ACR" --query id -o tsv)
az role assignment create --assignee-object-id "$PRINCIPAL_ID" --assignee-principal-type \
  ServicePrincipal --role AcrPull --scope "$ACR_ID"
az acr config authentication-as-arm update -r "$ACR" --status enabled
```

Role assignment requires Owner or appropriate role-assignment permission; Contributor alone is insufficient. If denied, ask the subscription administrator to assign AcrPull at this registry scope. RBAC propagation can take several minutes. If the registry uses ABAC repository permissions, AcrPull may not apply; use a normal RBAC-mode registry for these commands or follow the documented repository-reader setup.

7. Create the environment and deploy

```
az containerapp env create --name "$ENVIRONMENT" --resource-group "$RG" --location \
  "$LOCATION"
az containerapp create --name "$APP" --resource-group "$RG" \
  --environment "$ENVIRONMENT" --image "$LOGIN_SERVER/dengue-app:v1" \
  --user-assigned "$IDENTITY_ID" --registry-server "$LOGIN_SERVER" \
  --registry-identity "$IDENTITY_ID" --ingress external --target-port 8501 \
  --cpu 0.5 --memory 1Gi --min-replicas 0 --max-replicas 1
FQDN=$(az containerapp show -g "$RG" -n "$APP" --query \
  properties.configuration.ingress.fqdn -o tsv)
echo "https://$FQDN"
```

The environment uses Consumption workloads by default; confirm the workload choice in the portal. No GPU, VM, AKS cluster, Azure ML endpoint or database is needed. External ingress makes this demonstration publicly reachable. It only exposes public benchmark descriptors; do not add personal or confidential records.

8. Verify the live deployment and record evidence

1. Open the printed HTTPS URL. The first request after idle time can be slow because min replicas is zero. 2. Choose a sample and click Classify and explain. Test a second sample and a pasted public genome. Check that all sequence input, prediction, explanation and metrics appear. Do not describe model probability as calibrated confidence. 3. In portal search Container Apps > lanka-dengue-demo > Overview. Check running revision and application URL; inspect Log stream if startup fails. 4. In Terminal, inspect status/logs:

```
az containerapp show -g "$RG" -n "$APP" --query \
  '{State:properties.provisioningState,Revision:properties.latestReadyRevisionName}' -o \
  table
az containerapp logs show -g "$RG" -n "$APP" --tail 50
curl --fail "https://$FQDN/_stcore/health"
```

5. Save a browser screenshot with the URL, successful prediction, and date; a revision screenshot; and cost/credit evidence privately. Add the actual deployment URL and date to the report after successful verification. Local tests do not prove Azure works. 6. Compare results/metrics.json in the build logs or rebuild environment with the supplied report. Tiny numerical changes are possible between platforms. If split, best parameter or metrics change materially, regenerate the report before submitting.

9. Update and clean up

After editing source, use a new immutable image tag:

```
az acr build --registry "$ACR" --image dengue-app:v2 .
az containerapp update -g "$RG" -n "$APP" --image "$LOGIN_SERVER/dengue-app:v2"
```

After the marking window, delete ONLY this dedicated assignment resource group. First list its contents and confirm it contains no unrelated resources.

```
az resource list --resource-group "$RG" --output table
az group delete --name "$RG"
```

Read the interactive confirmation. Deletion removes the app, registry, identity, environment and logs in that group and makes the demo URL stop working. Save evidence first. Do not delete before a lecturer who needs the live link has marked it. Verify the group is gone in the portal; inspect any associated resources created elsewhere and continuing costs.

10. Troubleshooting and student build fallback

- Subscription not found: switch directory, sign into the same account used to activate student credits, then select the subscription again. - Location/RequestDisallowedByPolicy: inspect the error for allowed regions; use a permitted region for both registry and environment. Availability is subscription-specific. - Missing provider/extension: rerun registrations with --wait and upgrade the containerapp extension. - Registry name unavailable: use another globally unique lowercase alphanumeric name. - Image pull failure: check LOGIN_SERVER, tag v1, role assignment, ARM tokens, and propagation. Never print registry passwords as a workaround in a video. - Resource quota: inspect the requested quota and supported regions in the portal; do not create larger resources to bypass it. - Blank app/502: check logs, port 8501, server.address=0.0.0.0 and available memory. One GiB is an initial setting, not a measured peak guarantee. - Port 8501 busy locally: stop the old Streamlit process or choose `streamlit run app.py --server.port 8502` locally; Azure still uses 8501. - Serialized-model mismatch: run train.py with the pinned versions in your active venv. Load only trusted joblib files. - ACR Tasks unsupported: install Docker Desktop from https://www.docker.com/products/docker-desktop/ and start it. Keep the same registry and terminal variables, then run:

```
az acr login --name "$ACR"
docker build --platform linux/amd64 -t "$LOGIN_SERVER/dengue-app:v1" .
docker push "$LOGIN_SERVER/dengue-app:v1"
```

This fallback uses local CPU, works around cloud Tasks restrictions, and explicitly targets amd64 for Apple Silicon Macs. Resume at section 6. If ACR creation itself is blocked, this fallback cannot fix that; resolve subscription policy first.

11. Sources checked on 2 November 2026

Microsoft Azure for Students: https://azure.microsoft.com/en-us/free/students/ CLI installation: https://learn.microsoft.com/en-us/cli/azure/install-azure-cli Container Apps quickstart: https://learn.microsoft.com/en-us/azure/container-apps/get-started ACR cloud build: https://learn.microsoft.com/en-us/azure/container-registry/container-registry-quickstart-task-cli Managed identity image pull: https://learn.microsoft.com/en-us/azure/container-apps/managed-identity-image-pull CLI reference: https://learn.microsoft.com/en-us/cli/azure/containerapp Scaling: https://learn.microsoft.com/en-us/azure/container-apps/scale-app Budget behavior: https://learn.microsoft.com/en-us/azure/cost-management-billing/costs/tutorial-acm-create-budgets Container Apps pricing: https://azure.microsoft.com/en-us/pricing/details/container-apps/

GitHub repository and demonstration evidence

Repository: https://github.com/Thush20210156/lanka-dengue-sequence-classifier Open its Actions tab. The Reproduce and check workflow trains the model, validates all sequence-derived features and app submissions, then builds and health-checks the Docker image. A green run proves those CI checks, not an Azure deployment. After deployment record the public HTTPS URL, ready revision, application prediction and date. A local screenshot or CI screenshot is not Azure deployment evidence. Include 20-30 seconds of the GitHub source and commit history plus 30-40 seconds of the actual Azure app in your final 3-5 minute video. Show the accurate commit history; do not alter author dates to suggest earlier work.